

8 BIT BREADBOARD COMPUTER - MODULES

This documentation consists of the working details of each individual modules of an 8 bit breadboard computer. This computer is referenced from the works of Ben Eater^[1].

1. CLOCK MODULE

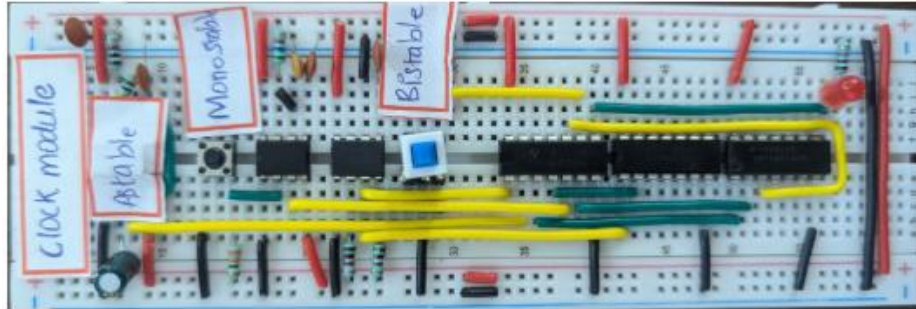


Figure 1: Clock module on breadboard

The purpose of a clock in a computer is to synchronise the operations that takes place in the computer. The clock module of this computer is constructed using 555 timer IC. This IC has three states: astable, monostable and bistable. These three states along with some combinational logic makes the clock module.

1.1 Astable mode

The astable mode of the 555 timer provides an astable square waves as output. To have an adjustable frequency, a variable resistor can be used in series. Refer appendix A.1 for working of astable multivibrator.

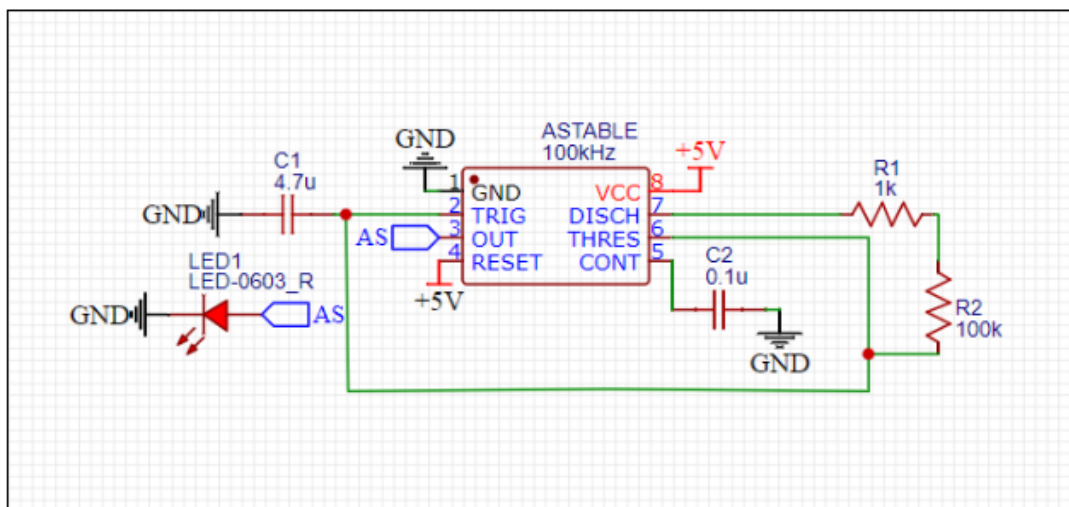


Figure 2: Circuit for Astable mode

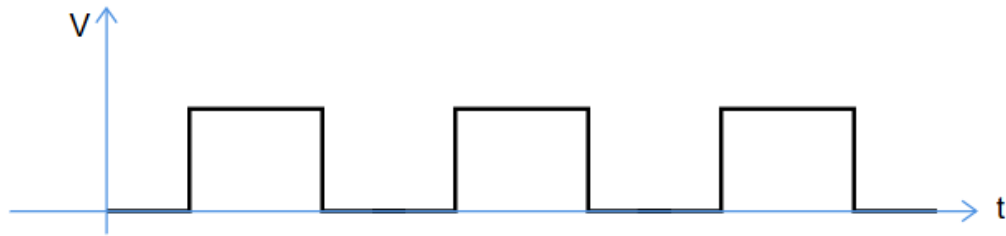


Figure 3: *Astable clock pulse*

1.2 Monostable mode

A simple pushbutton will provide us with monostable pulse, but the movable contact point (illustrated in figure 4) in a push button might have some bouncing effects. Having the bouncing effect in a simple circuit is insignificant but the bouncing effect will have much significance when it is used as a clock pulse in a computer. The main purpose of using the monostable pulse in this computer is for debugging, if there is a bouncing effect, it may skip a few cycles which will affect the debugging process. So 555 timer is used as debouncer circuit for the pushbutton. Refer appendix A.2 for working of monostable multivibrator.



Figure 4: *Pushbutton working*^[2]

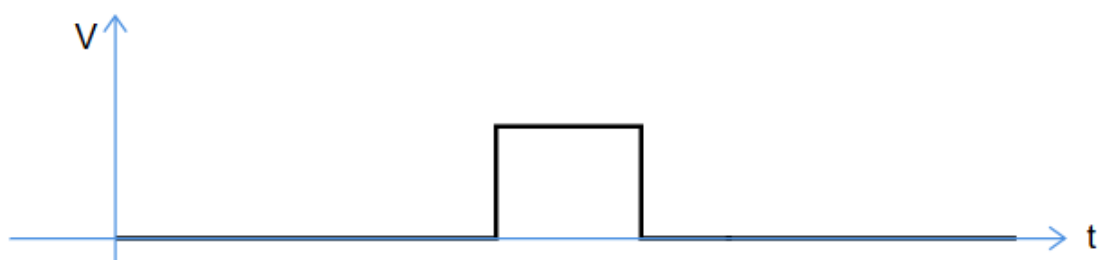


Figure 5: *Monostable clock pulse*

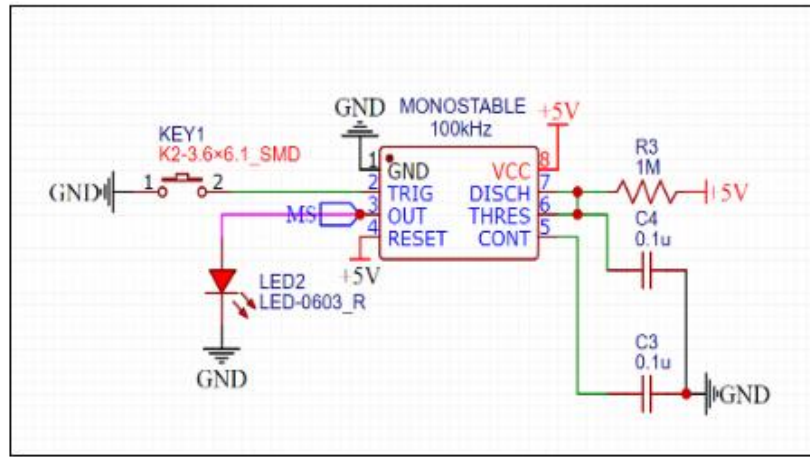


Figure 6: Circuit for monostable mode

1.3 Bistable mode

A double throw switch is used here to switch between the astable and monostable, it essentially acts as select line and the 555 timer is used to build the debouncer circuit. Refer appendix A.3 for the working of bistable multivibrator.

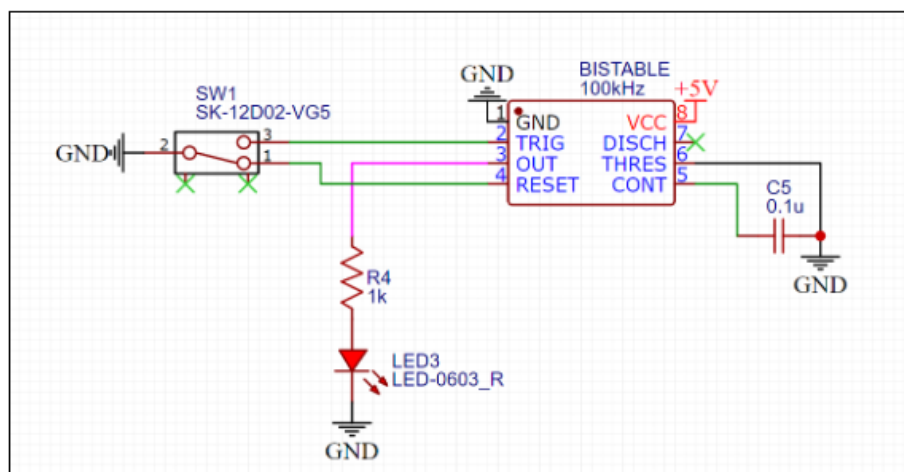


Figure 7: Circuit for Bistable mode



Figure 8: Bistable clock pulse

1.4 Combinational Logic

A 2:1 multiplexer is used to switch between the astable mode and monostable mode with bistable pulse as the select line. A halt signal which will come from the program of the computer is anded with the clock pulse in case the clock needs to be halted.

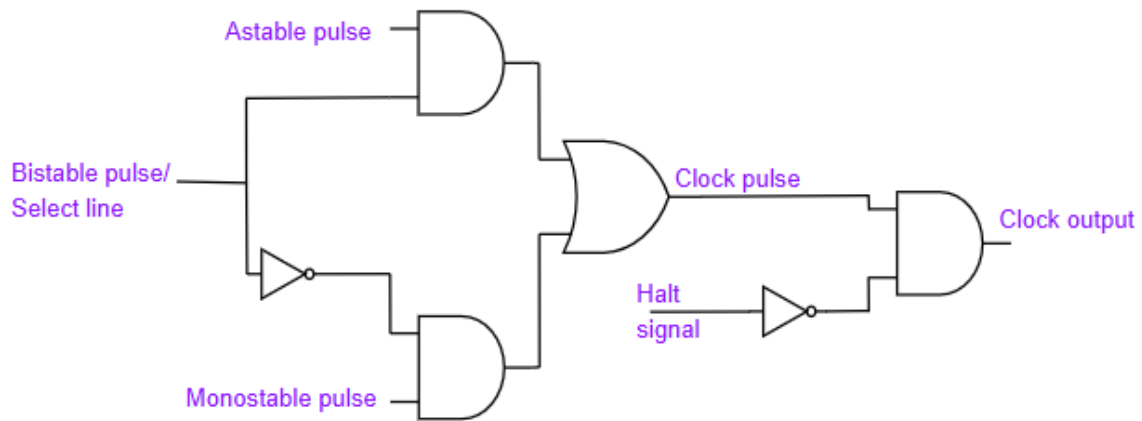


Figure 9: Selection circuit for the clock

1.5 Bill of Materials

S. No	Component Name	Specification	Quantity
1	BreadBoard		1
2	555 timer	LM555	3
3	Hex inverter	74LS04	1
4	Quad AND gate	74LS08	1
5	Quad OR gate	74LS32	1
6	Ceramic capacitor		5
7	Electrolytic capacitor		1
8	Resistor		7
9	DPDT switch		1
10	Push button		1
11	LED		1
12	Connecting Wires	As required	

Table 1: Clock Module BOM

2. REGISTER A, REGISTER B, INSTRUCTION REGISTER

A register in a computer is a small and fast storage that is directly accessible to the processor and stores the data or instruction during execution. In this computer we have 3 registers namely Register A, Register B and Instruction register. A and B are general purpose registers and Instruction register (some books also refers to this as current instruction register CIR) stores the instruction that is currently being executed.

2.1 Tristate Logic and its need

In the IC's we use, there are only two states when they are powered on, high and low. These states are implemented using a transistor which sources or sinks current., In case of high, the transistor Q2 is turned on, which sources the current. In case of low, the transistor Q3 is turned on, which sinks the current.

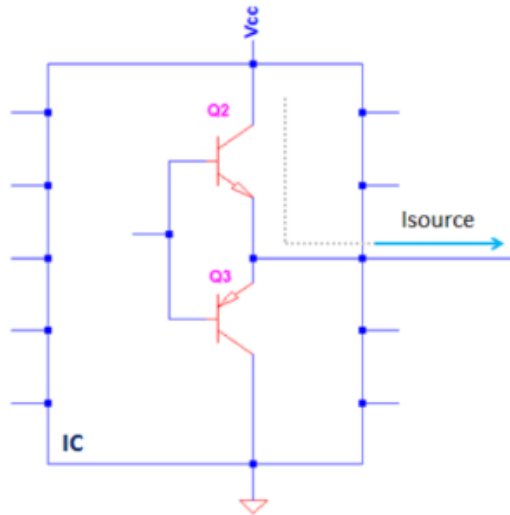


Figure 10: Sinking and Sourcing current in ICs^[3]

When registers are connected to the bus, if one module is outputting high and other is outputting low, the sourced current of one register will be sunk by other, causing inconsistencies in data produced. For this tristate logic is implemented. The three states are high, low and high-impedance state. The high-impedance state electrically disconnects the device from the circuit. So a tristate buffer is used along with the register ICs, which will make the ICs either to output/input the data or be disconnected.

2.2 Register and connection with bus

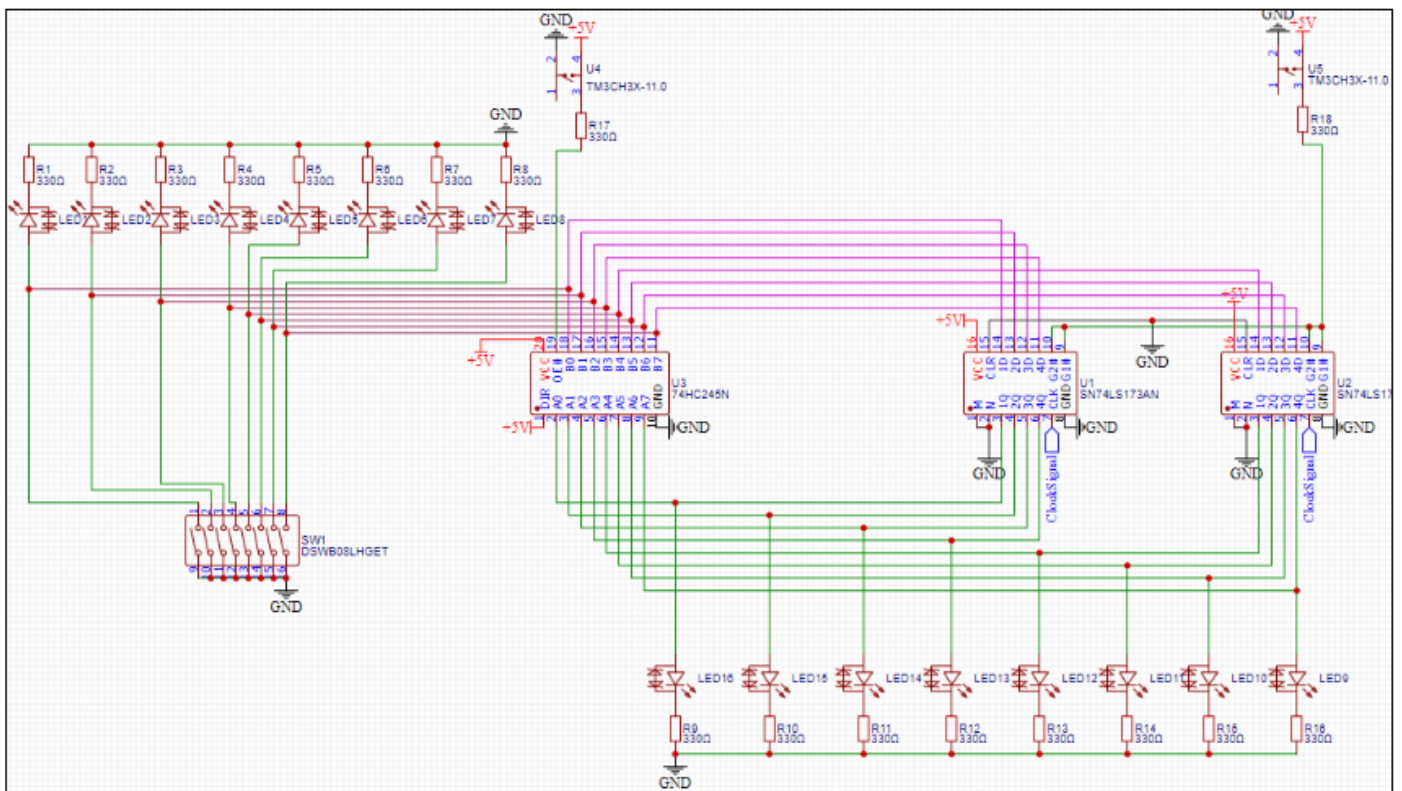


Figure 11: Circuit for register with bus

CONNECTIONS:

- The 74LS173 ICs are Registers with tristate logic, so one could use it directly without the buffer, but in my circuit, I connected the output of the registers with LED's to display what's in the register at all times, and used a tristate buffer to connect them to a bus.
- For the above operation the M and N pins of the Registers are grounded.
- The 74LS173 ICs can store 4 bits, but we require a width of 8 bits, so 2 ICs have been used.
- The G1 and G2 pins are connected to a pull up resistor for LOAD operation.
- The output pins of registers are connected to respective A pins in the buffer and the respective B pins are connected to the bus. The DIR pin is set high, since we are utilising only one direction of the bidirectional tristate buffer^[5], and the direction is from A pins (connected to the registers) to B pins (connected to the bus).
- We have also connected the bus with the input of register.
- The series of LEDs in the above circuit essentially acts as a bus.

In the above circuit, since it is built as a separate module, a 8 way DIP switch has been used to control the bits that can be fed into the registers, in case of the whole computer, one could just connect it to the bus directly, while each line of the bus is connected to ground via 10k Ω resistor (This is done to make the data in bus 0 when no module is connected to the bus).

These registers ICs are made up of D flip flops, refer to appendix B1 for its working.

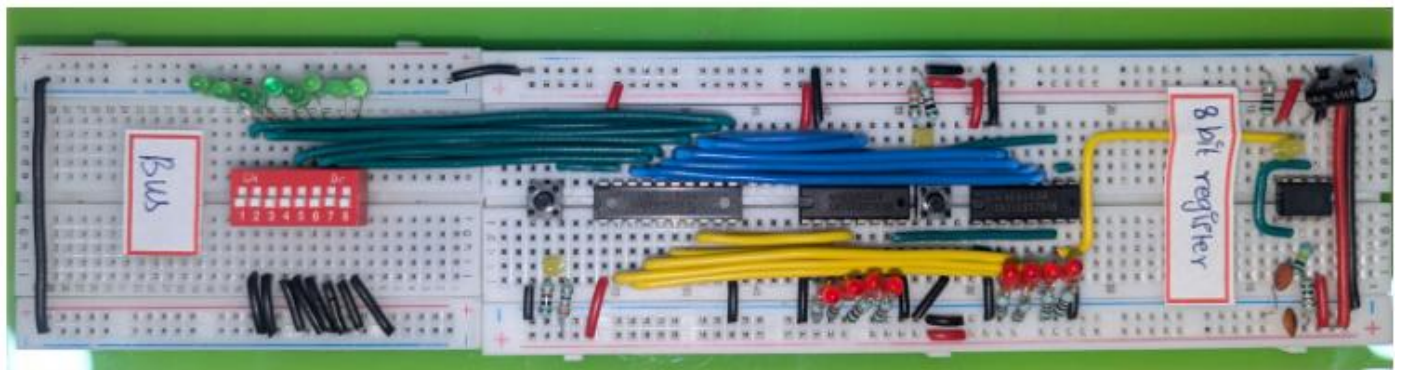


Figure 12: Register module on a breadboard

The general purpose registers(A,B) have all the 8 bits connected to the bus with both load (BUS to REGISTER) and enable (REGISTER to BUS) functionalities. In the IR, all 8 bits can be loaded from the bus, whereas only the 4 LSB (Least Significant Bit) bits are outputted to the bus and the MSB(Most Significant Bit) 4 bits are not outputted to the bus and are connected to the Control unit of this computer.

2.3 Bill of Materials

S. No	Component Name	Specification	Quantity
1	BreadBoard		1
2	Mini Breadboard		1
3	4-bit D register	74LS173	2
4	555 timer	LM555	1
5	Octal bus tansceiver	74HC245	1
6	Ceramic capacitor		2
7	Electrolytic capacitor		1
8	Resistor		15
9	Push button		2
10	8 way DIP switch		1
11	LED		19
12	Connecting Wires	As required	

Table 2: *Register BOM*

3. ARITHMETIC AND LOGIC UNIT (ALU)

The arithmetic and logic unit in a typical computer can perform arithmetic, bitwise and comparison operations. In this breadboard computer, the ALU can perform addition and subtraction only. Two 4 bit adder ICs 74LS283^[6] are cascaded for the ALU to perform addition of 8 bit numbers. The subtraction is done using the addition of negative number represented in 2's complement.

The XOR gate flips the first input when the second input is HIGH. Using this logic, XOR gate is used to find the one's complement of a number. For the two's complement(one's complement + 1), we are setting the carry over pin of the adder to high when subtracting. The second inputs of the XOR gate and the carry over pin of the adder IC is connected to a push button to indicate whether we are adding or subtracting. The output of the ALU is then connected to the bus via a tristate buffer.

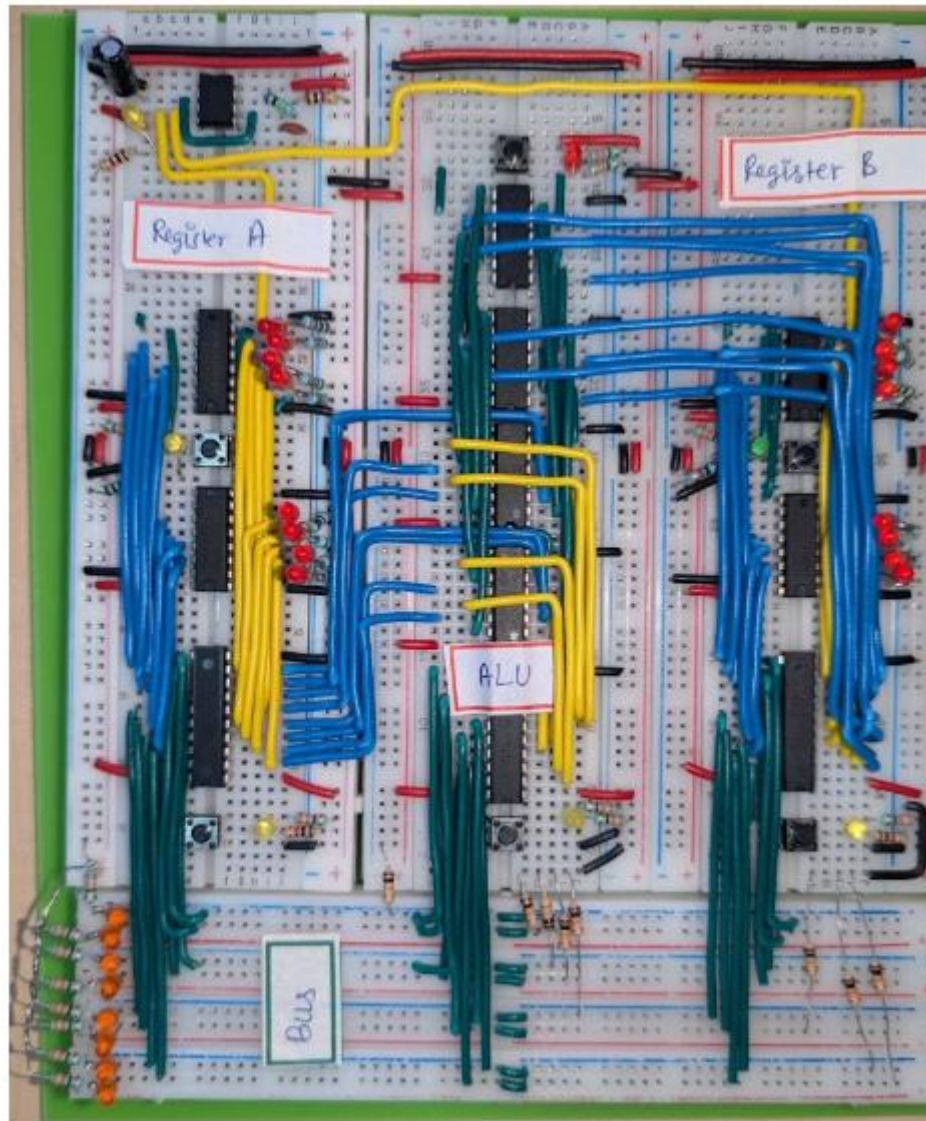


Figure 14: ALU module on a breadboard

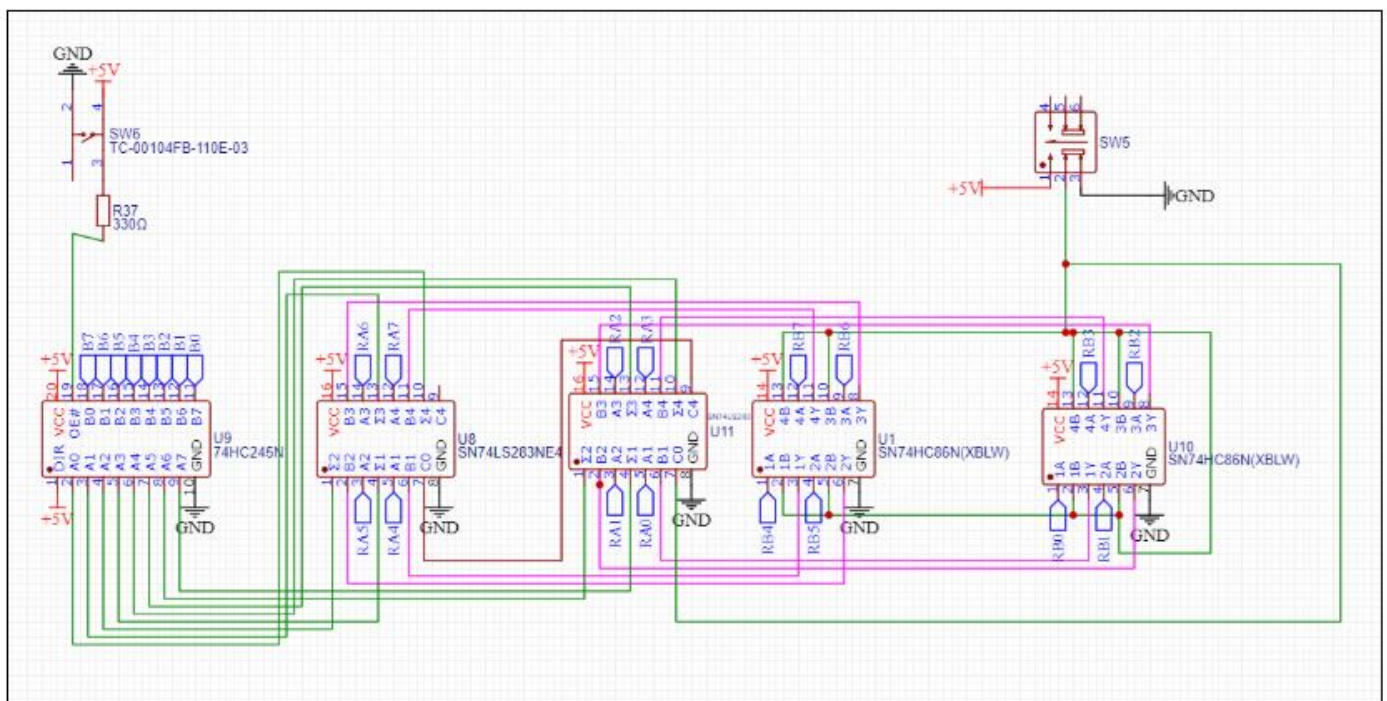


Figure 15: Circuit diagram of ALU

CONNECTIONS:

- The A pins of binary adder ICs (74LS283) are connected to the Q (output) pins of Register A.
- The B pins are connected to the outputs of XOR gate.
- The A inputs of XOR gates are output pins of Register B and B inputs of XOR gates is connected to the SUB switch which decides whether we are adding or subtracting at the moment. When SUB is low → addition, SUB is high → subtraction.
- The output of adder ICs are connected to the A pins of buffer IC.
- The carry pins of the two adder ICs are connected to cascade them to form 8 bit adder.
- The B pins of buffer IC is connected to the bus and the direction pin is set high to set the direction from A to B.
- In the above diagram, B(0 - 7) represent the bus line, RA(0 - 7) represent the outputs of Register A, and RB(0 - 7) represent the outputs of Register B.

3.1 Bill of Materials

S. No	Component Name	Specification	Quantity
1	BreadBoard		4
2	4-bit D register	74LS173	4
3	555 timer	LM555	1
4	Octal bus tansceiver	74HC245	3
5	4-bit adder	74LS283	2
6	Quad XOR gate	74LS86	2
7	Ceramic capacitor		1
8	Electrolytic capacitor		1
9	Resistor		47
10	Push button		6
11	LED		31
12	Connecting Wires	As required	

Table 3: ALU BOM

4. RANDOM ACCESS MEMORY (RAM)

The RAM will contain the program (set of instructions) that is being currently being executed. The size of RAM used in this computer is 16 Bytes, with only 4 address bits, which will reduce the complexity of the programs executed in this computer. If it had more RAM, just with the addition and subtraction functionality, one could do complex operations. But the small RAM size is one of the downsides of this computer.

Referring to the one-bit register discussed in Appendix B.1, it includes write, enable, input, and output pins. The corresponding block diagram is illustrated as follows:

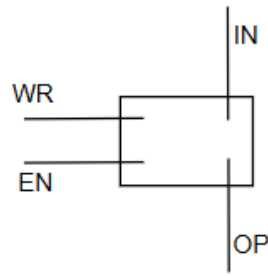


Figure 15: Block diagram of 1 bit register

By cascading 8 one bit registers, we'd get a 8 bit register that is used in this computer where the WR and EN pins of each one bit register is tied together and it would look like as follows:

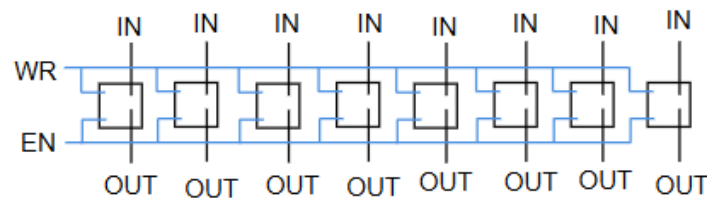


Figure 16: Block diagram of 8 bit register

As mentioned before, this RAM has 4 address lines, so if each block has 8 bits(word length), we can have $2^4 = 16$ blocks which are addressable (0 to 15), so this stores a total of 128 bits or 16 bytes.

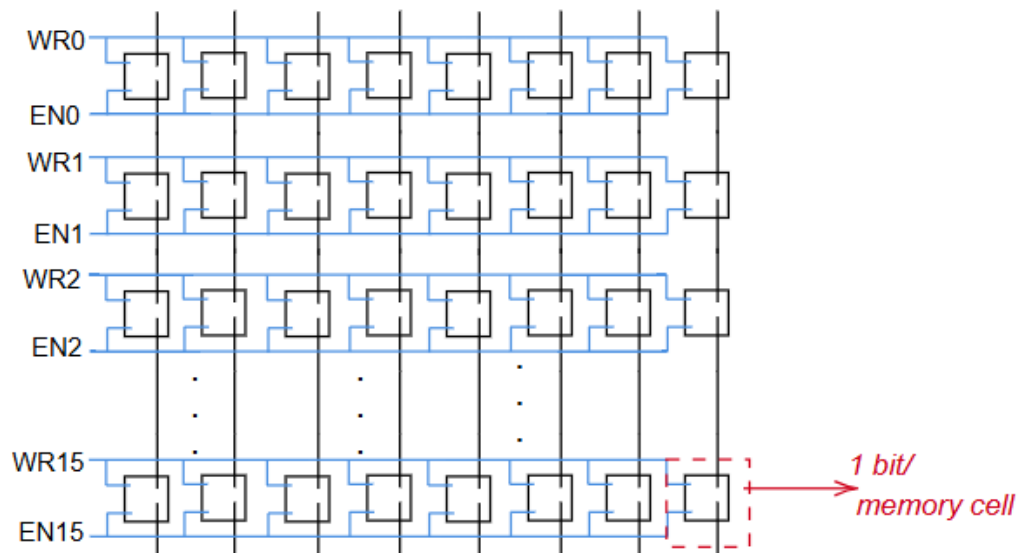


Figure 17 : Block diagram of 16 byte RAM

There are two types of RAM, SRAM (static RAM) and DRAM (dynamic RAM). In DRAM, the data is stored as charges in capacitors. But the charges in the capacitors will discharge slowly, and will lose the data, for this the data is refreshed frequently using an external memory refresh circuit which periodically rewrites the data, hence the name dynamic RAM. Even when there is overhead due to this refreshing, it has some advantages when compared to SRAM , like the simplicity of the memory cells. In static RAM, the data is stored in flip flops,

and doesn't require any refreshing, the disadvantage is the complexity of a circuit required for a memory cell when compared to DRAM. In this computer, SRAM is used, since it doesn't require a refreshing circuit.

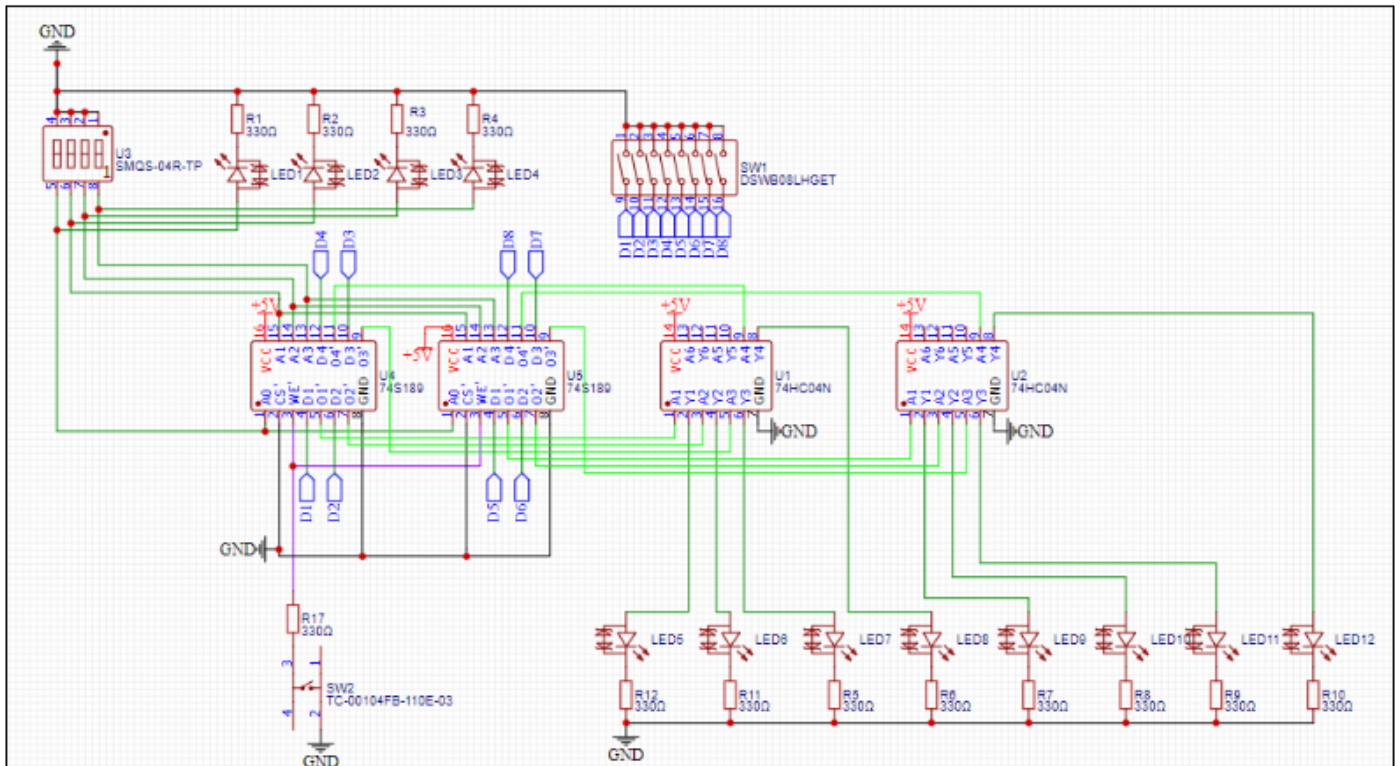


Figure 18: Circuit diagram of RAM

CONNECTIONS:

- The IC used for RAM is SN74S189N which has a width of 4 bits, so two ICs are cascaded to get 16 bytes.
- The 4 address pins are tied together and are connected to the 4 way switch (The address pins are pull up pins, so they are connected to ground normally and when switched, they are disconnected from the ground).
- The 8 data pins are connected to the 8 way switch and is connected to ground in the same way as the address pins.
- The output pins give the inverted data as output so they are fed into a not gate to get the actual data which are indicated via LEDs.
- The Chip Select pins are active low pins and are connected to ground.
- The Write Enable pins are also active low pins and are connected to a pull-up switch button.

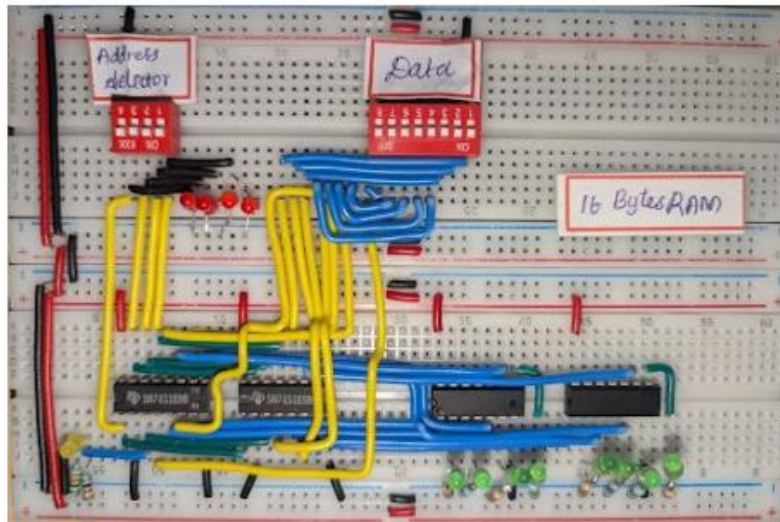


Figure 19: RAM module on breadboard

When the above module is integrated with the computer, the outputs of NOT gates are connected to a tristate buffer then to the bus. We can say that the above design is for when the computer is in "programming mode" where the data in RAM is manually written. When the computer is in "operating mode" the data input and the address input could be from any of the other modules which will be received via bus. To communicate with the bus, the data from the bus is connected to a tristate buffer and is multiplexed with the 8 way switch(manual data for RAM) and a select switch(which decides whether the computer is in programming or operating mode). The output of the multiplexer is given as input to the data pins of RAM.

4.1 Bill of Materials

S. No	Component Name	Specification	Quantity
1	BreadBoard		2
4	64-bit RAM	74189	2
5	Hex inverter	74LS04	2
7	8 way DIP switch		1
8	4 way DIP switch		1
9	Resistor		10
11	LED		13
12	Connecting Wires	As required	

Table 4: RAM BOM

5. MEMORY ADDRESS REGISTER (MAR)

MAR can also be called as address buffer. It holds the address location in which the data needs to be stored or retrieved. A separate MAR module is not done because the RAM isn't integrated to the computer. In case the RAM is integrated to the computer, as discussed in the above section, when in programming mode, the address and data are entered manually. When it is in operating mode, the data and the address for the RAM will be from bus. How data is inputted is discussed in the above section. For the address, the 4 way

The 4 way switch imitates the functionality of the bus in the computer. The DPDT switch acts as the enable for counting.

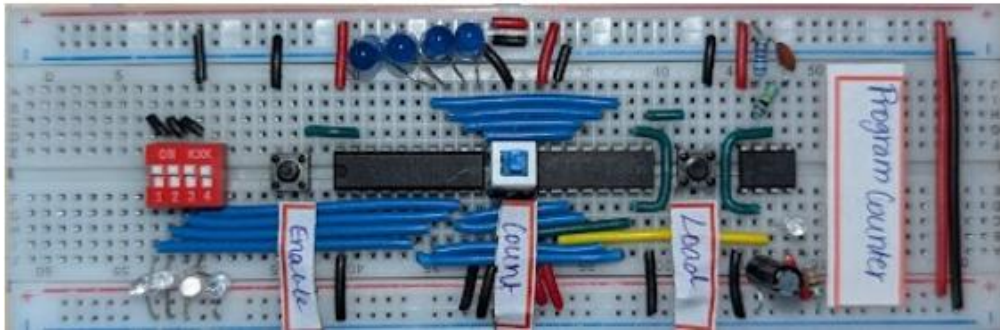


Figure 21: *Program Counter on breadboard*

6.1 Bill of Materials

S. No	Component Name	Specification	Quantity
1	BreadBoard		1
2	4-bit binary counter	74LS161	1
3	555 timer	LM555	1
4	Octal bus tansceiver	74HC245	1
5	4 way DIP switch		1
6	DPDT switch		1
7	Ceramic capacitor		1
8	Electrolytic capacitor		1
9	Resistor		3
10	Push button		2
11	LED		9
12	Connecting Wires	As required	

Table 5: *Program Counter BOM*

7. OUTPUT REGISTER

An output register in a typical computer is a register that stores the output of a computation. In this computer, the output register does the same and is similar to Register A and B, but instead of displaying the output in binary that is hard to read, seven-segment displays are used to display in decimal. The conversion of binary to a seven-segment display requires the following combinational logic, and the implementation of the same using basic logic gates requires a complex setup in a breadboard, or a separate decoder IC for each display is required.

Binary Inputs				Decoder Outputs							7-Segment Display Outputs
D	C	B	A	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	1	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	1	0	1	1	9

Figure 23: Truth table for binary to common cathode seven-segment display^[7]

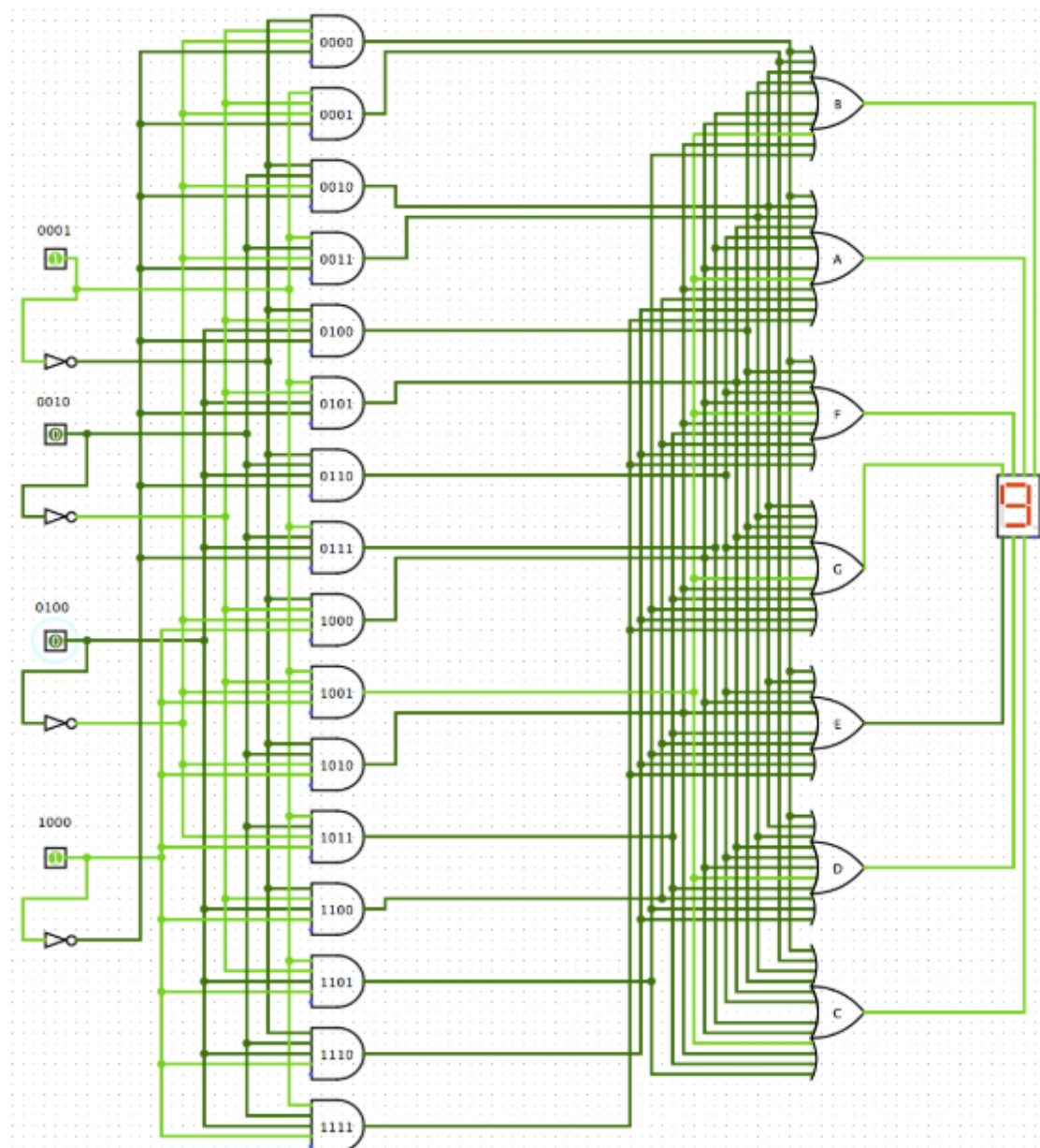


Figure 24: Combinational logic for binary to seven-segment display^[8]

In digital electronics, there are two types of logic, combinational and sequential. In combinational logic, only the current inputs constitute the current output of the system. In sequential logic, the output is dependent on the current inputs(current state) and also the previous state. Every combinational logic circuit can be replaced by an EEPROM(Electrically Erasable Programmable Read-Only Memory). 28C16^[9] IC is used here. The conversion of binary to seven-segment display required for the output register can be programmed in the EEPROM. The address pins of the EEPROM can be set to the current input and the data in the address can be programmed to be the output of the required combinational logic.

7.1 EEPROM programmer

A programmer available in the market can be used to program the EEPROM, but this project uses Arduino Nano for programming. Since Arduino Nano doesn't have enough digital pins, shift registers are used to compensate for it. We are using two shift registers - 74HC595D. The address and OE pins of EEPROM is connected via shift register and Input and write enable pins of EEPROM is directly connected to Arduino Nano.

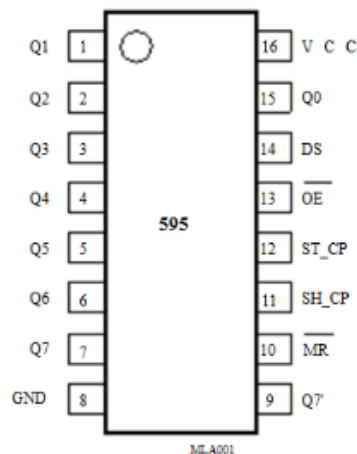


Figure 25: 74HC595D IC pin out^[9]

The connection from Arduino to EEPROM IC is as follows:

- EEPROM I₀ to I₇ is connected to Arduino D₅ to D₇ and $\overline{WE}$ is connected to D₁₃
- DS of the shift register to Arduino D₂, SHCLK to D₃ and STCHK to D₄
- EEPROM A₀ to A₇ is connected to 1st shift register's Q₀ to Q₇. A₈ to A₁₀ is connected to second shift register's Q₀ to Q₂ and $\overline{OE}$ to second shift register's Q₇.

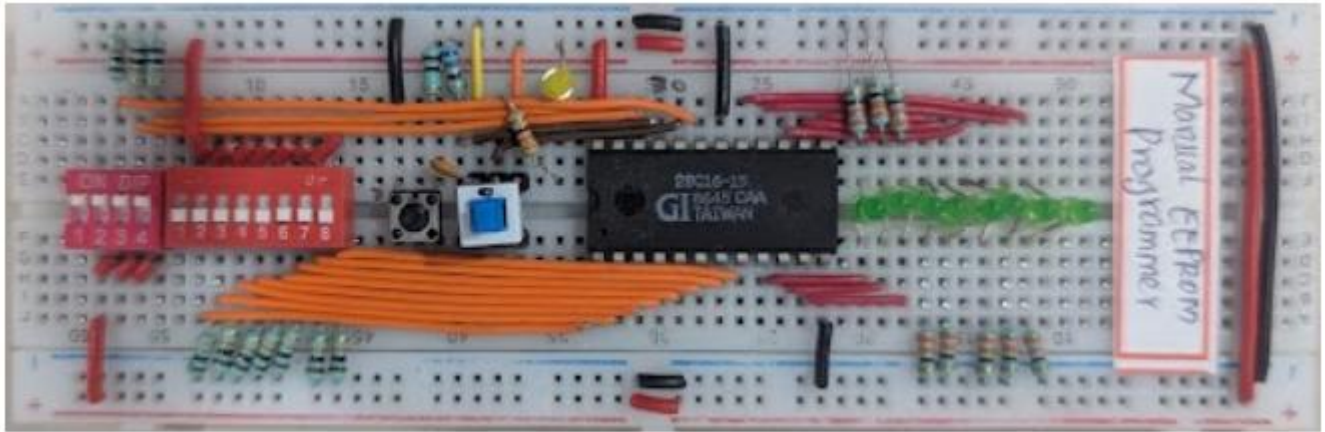


Figure 26: *Manual EEPROM programmer on Breadboard*

The manual EEPROM programmer in the above image doesn't have Arduino and is used to visualise the programming of EEPROM.

The code for programming the EEPROM is same as Ben Eater's^[11].

7.2 Multiplexing the Displays

Since our computer handles 8 bit data, the maximum value of integer when using unsigned data representation is 255, so three displays can be used with an EEPROM for each display. That is a more straightforward approach. But instead of using three EEPROM ICs and only using the first 256 addresses in each EEPROM to display the one's, ten's and hundred's place, in a single EEPROM, the first 256 addresses for one's place, second 256 addresses for ten's place, third 256 addresses for hundred's place and the next three sets of 256 addresses can be used for signed magnitude representation when 4 displays are used.

For the above logic, A0 to A7 pins of EEPROM are from bus (using a register for integration). A8, A9 as a place decider (choosing which place to display) and is taken as input from 2 bit counter. The common ground pin of each display is connected to decoder output. This decides which display to turn on. Since the decoder's input is the same counter, when one's place is displayed, corresponding display will turn on. There will be flickering effect for a slow clock rate. Using a separate clock with higher rate (Output of the clock can't be flickering), as the clock for the counter will remove the flickering effect and the multiplexed display will output the number which can be perceived as a single number. The code for programming the EEPROM is same as Ben Eater's^[12].

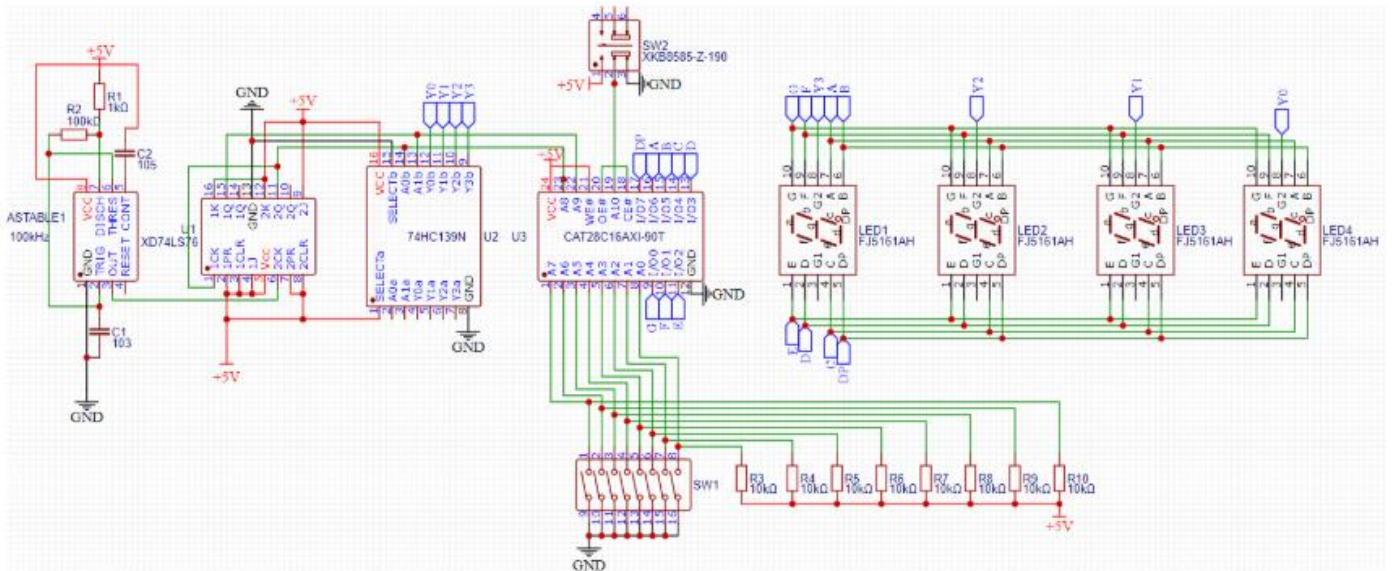


Figure 27: Circuit diagram of Output Register

The 8 way switch imitates the functionality of the bus in the computer from which the data to be displayed is given as address to the EEPROM.

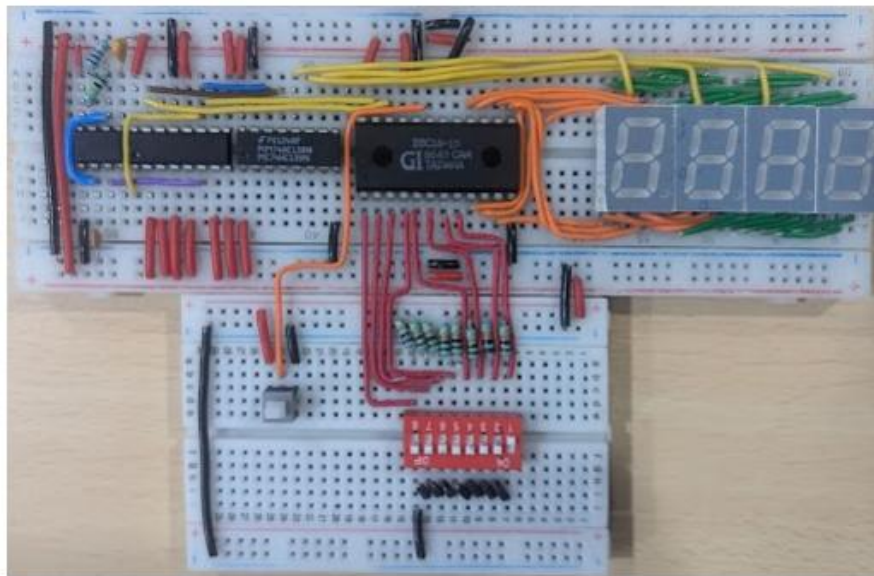


Figure 28: Output Register on Breadboard

7.3 Bill of Materials

Output Register

S. No	Component Name	Specification	Quantity
1	BreadBoard		1
2	Mini Breadboard		1
3	555 timer	LM555	1
4	Dual 2-to-4 line decoder/demultiplexer	74LS139	1
5	Dual J/K flip-flop	74LS76	1
6	16K EEPROM	28C16	1
7	8 way DIP switch		1
8	DPDT switch		1
9	Ceramic capacitor		2
10	7 Segment Display	Common Cathode	4
11	Resistor		10
12	Connecting Wires	As required	

Table 6: Output Register BOM

Manual EEPROM programmer

S. No	Component Name	Specification	Quantity
1	BreadBoard		1
2	16K EEPROM	28C16	1
3	8 way DIP switch		1
4	4 way DIP switch		1
5	DPDT switch		1
6	Push Button		1
7	Ceramic capacitor		1
8	LED		9
9	Resistor		21
10	Connecting Wires, Jumpers	As required	

Table 7: Manual EEPROM programmer BOM

8. CONTROL UNIT

The control signals from each module is collectively controlled and displayed using LEDs in a separate breadboard. The active high pins are directly connected whereas the active low pins are connected via a NOT gate to make the entire logic active high. The signals are connected with LEDs in the following order.

HLT	MI	RI	RO	IO	II	AI	AO	EO	SU	BI	OI	CE	CO	J
-----	----	----	----	----	----	----	----	----	----	----	----	----	----	---

Control Signals Legend:

HLT - Halt

MI - MAR in

RI - RAM in

RO - RAM out

IO - IR in

II - IR out

AI - A register in

AO - A register out

EO - Sum out

SU - Subtract

BI - B register in

OI - Output Register in
CE - Count Enable (increment PC)
CO - Counter out
J - Jump (PC in)

The instructions must be uploaded in RAM before execution. In this computer, the instructions are manually uploaded into RAM. The following is an example program:

RAM ADDRESS	Mnemonic (Assembly)	Opcode (Assembled Machine Language)
0000	LDA 14	0001 1110
0001	ADD 15	0010 1111
0010	OUT	1110 0000

•
•
•

1110	--- (data)	0001 1100 (28)
1111	--- (data)	0000 1110 (14)

Here, In the opcode for instructions:

4 bits - Instruction	4 bits – Address/ Data (operand)
----------------------	----------------------------------

In the opcode for data;

8 bits - Data

The Mnemonic doesn't need to be the same as above since it defined as part of system's design and implementation.

LDA XX - Load contents in XX from RAM into A register
 ADD XX - Add the contents in XX from RAM to A register
 OUT - No operands, Output the contents in A register

Every instruction has micro-instructions to be executed for the instruction to complete, and it begins with fetch cycle.

Micro Instructions for LDA 14:

CO MI ----- T0
RO II CE ----- T1
IO MI ----- T2
RO AI ----- T3

Each line represents a single clock cycle and the active signals.

A 4 bit binary counter (74LS161) is used to keep track of which micro-instruction the computer is currently executing. The clock of the computer is inverted and given as input to the counter so that the micro-instruction setting is done before executing the same. A 3:8 decoder (74LS138) is used to indicate the same and Y5 is connected to CLR of the counter via the master reset logic to reset the counter.

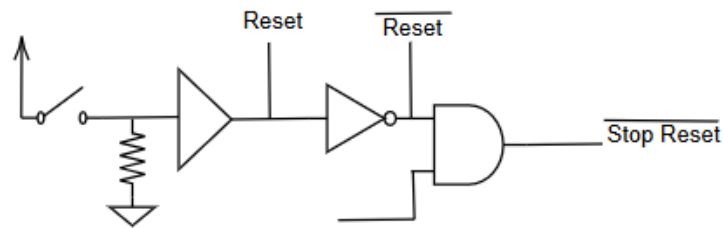


Figure 28 :Master Reset Logic

For the example program, the truth table with Instructions and step as input and control signals as output is given below. The steps are up to 4, because the counter is reset at step 5.

[illegible]

The above logic is just a simple truth table and can be implemented using simple logic gates. To simplify the implementation of the above combinational logic, two EEPROMs are used. To optimise the program, the MSB EEPROM's A7 is connected to *GND* and LSB EEPROM's A7 is connected to *Vcc*. The code for programming the EEPROM is same as Ben Eater's^[13].

The possible instructions with current hardware of the computer are:

- NOP - No operation
- LDA - Load contents in RAM at specified address into A register
- LDI - Load immediate data into A register
- HLT - Halt the clock
- ADD - Add contents in RAM at specified address to A register
- SUB - Subtract contents in RAM at specified address from A register
- JMP - Jump to the specified instruction at specified address in RAM
- STA - Store the contents in A register in RAM at specified address
- OUT - Output the contents in A register

One can also note that there are redundant cycles for certain instructions and those can be optimised.

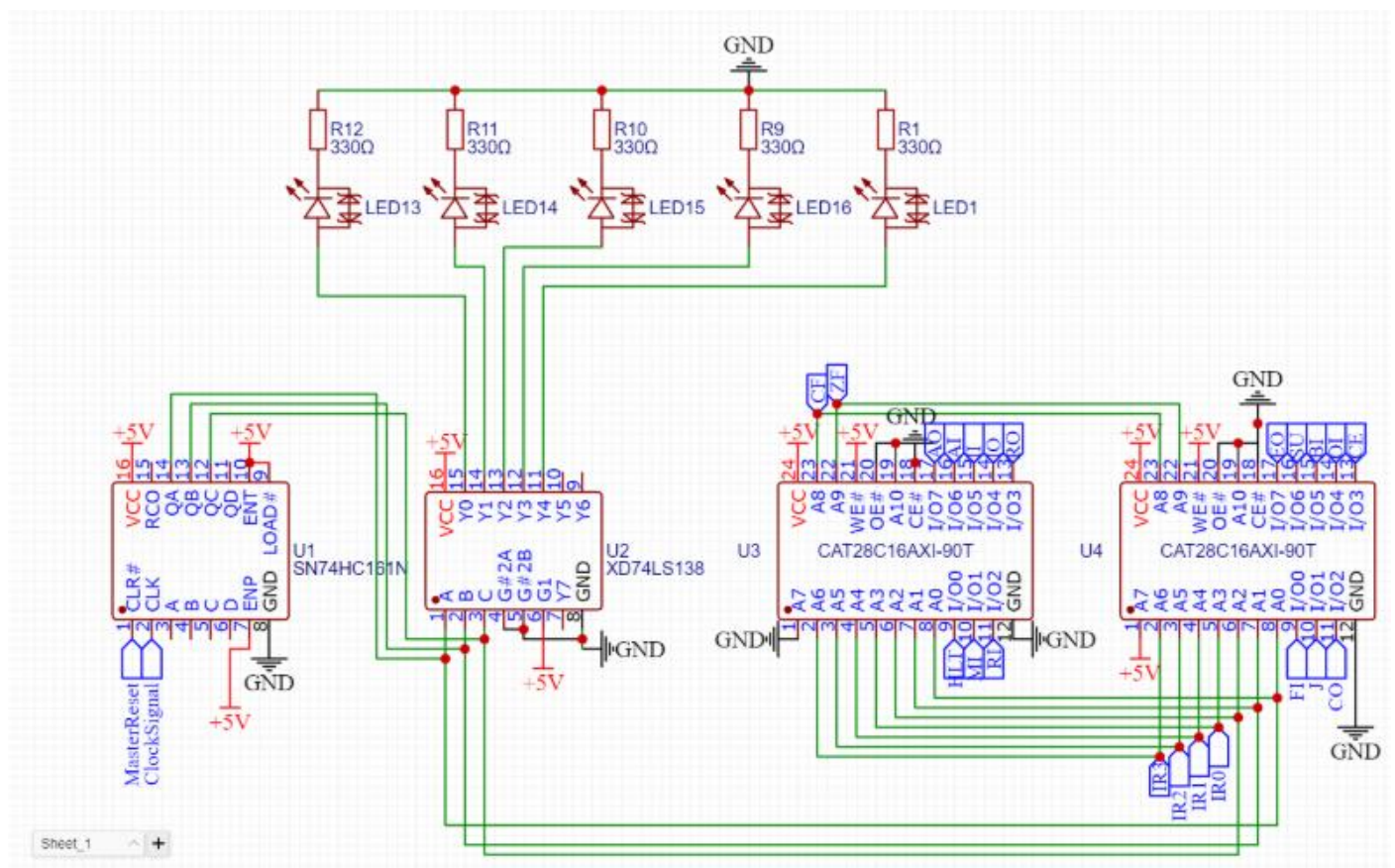


Figure 29: Circuit diagram of Control Unit

8.1 Achieving Turing Completeness

This 8 bit breadboard computer is aimed to be a general purpose computer. A general purpose computer is supposed to do every computation. But what is everything computable? The things which are computable is defined by Church-Turing thesis^[14] which defines that something is computable if and only if it can be computed by a Turing machine.

Currently with the available instructions mentioned in the above section, this is not a computer. It needs conditional jump instructions to be Turing complete. Conditional jump instructions implemented in this computer are JZ (Jump if Zero) and JC (Jump if Carry). Implementing these instructions require additional hardware.

The carry out from MSB adder in ALU is taken as carry flag. Each bits of ALU's output isanded to get the zero flag. In the ADD and SUB instructions, since the A register is updated after operation, the carry flag and zero flag may change so they are stored in a separate register called flag register, and its load signal is taken as another control signal as FI (flag in). Now the available control signals are:

HLT	MI	RI	RO	II	AI	AO	EO	SU	BI	OI	CE	CO	J	FI
-----	----	----	----	----	----	----	----	----	----	----	----	----	---	----

To implement the decoding, the carry flag is connected to EEPROM's A8 and zero flag to A9.

Instruction	Micro-instructions
NOP [No operation - 0000]	CO MI RO II CE - - -
LDA [Load A reg - 0001]	CO MI RO II CE IO MI RO AI -
ADD [Add to A reg - 0010]	CO MI RO II CE

	IO MI RO BI EO AI FI
SUB [Subtract from A reg - 0011]	CO MI RO II CE IO MI RO BI EO AI SU FI
STA [Store contents in A reg - 0100]	\table{CO MI RO II CE IO MI AO RI -
LDI [Load immediate data in A reg - 0101]	CO MI RO II CE IO AI - -
JMP [Jump to given instruction - 0110]	CO MI RO II CE IO J - -
C [Jump if carry - 0111]	CO MI RO II CE -

	IO J (If CF =1, else no active signals) -
JZ [Jump if zero - 1000]	CO MI RO II CE - IO J (If ZF =1, else no active signals) -
OUT [Output contents in A reg - 1110]	CO MI RO II CE AO OI - -
HLT [Halt - 1111]	CO MI RO II CE HLT - -

Each instruction takes 5 clock cycles. Each line of the micro-instruction represents a clock cycle and the active instructions. The code to program the EEPROM with flags functionality is same as Ben Eater's ^[15].

6. BUS

The entire architecture of the 8 bit compute is given below.

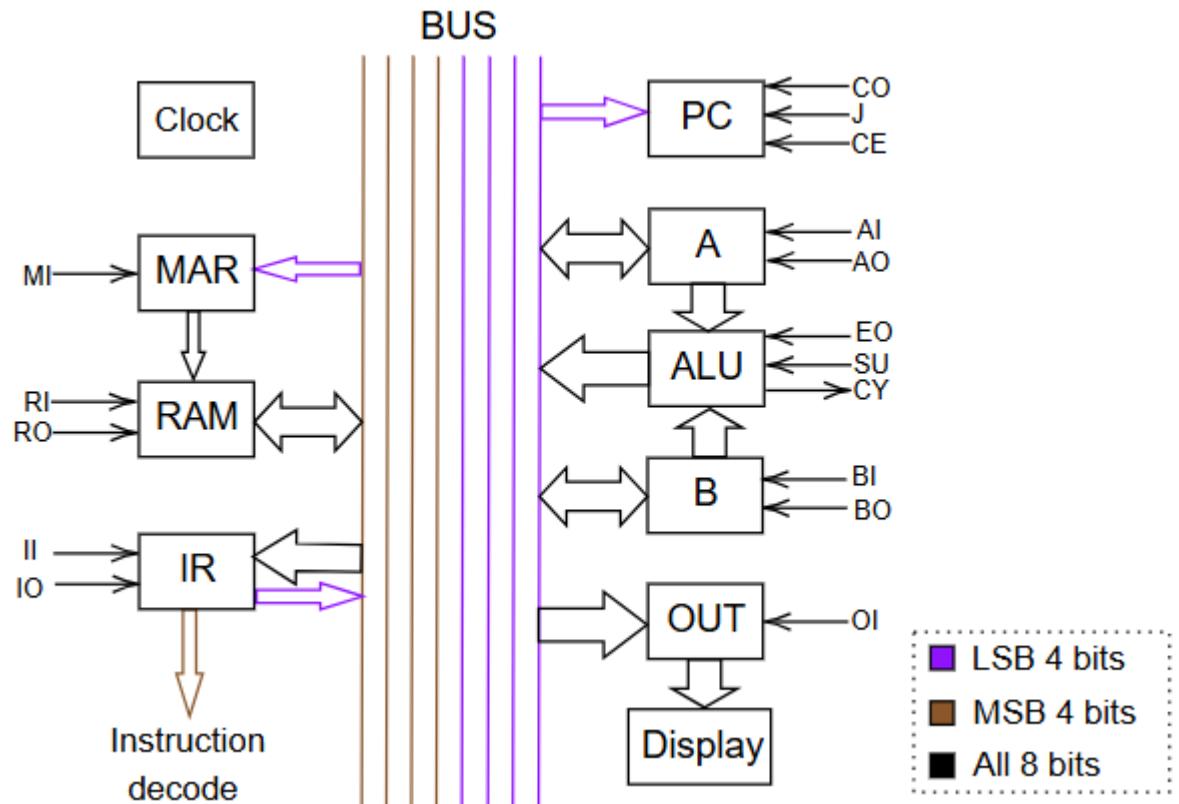


Figure 17 :Architecture of the 8 bit computer

APPENDIX

A.1 Astable multivibrator working

When it comes to digital logic and digital circuits we are dealing with two-state devices consisting of the states 0 and 1. Multivibrator is an electronic circuit that is used to implement such two-state devices (oscillators, timers, flipflops etc.,). An oscillator is a device that produces periodic signals such as square waves. NE555 timer in astable mode works as a multivibrator oscillator switching between two states with neither of the states being stable hence the name astable. The 555 timer IC internally consists of a SR latch, two magnitude comparators and a voltage divider network consisting of three 5k Ω resistors, making the

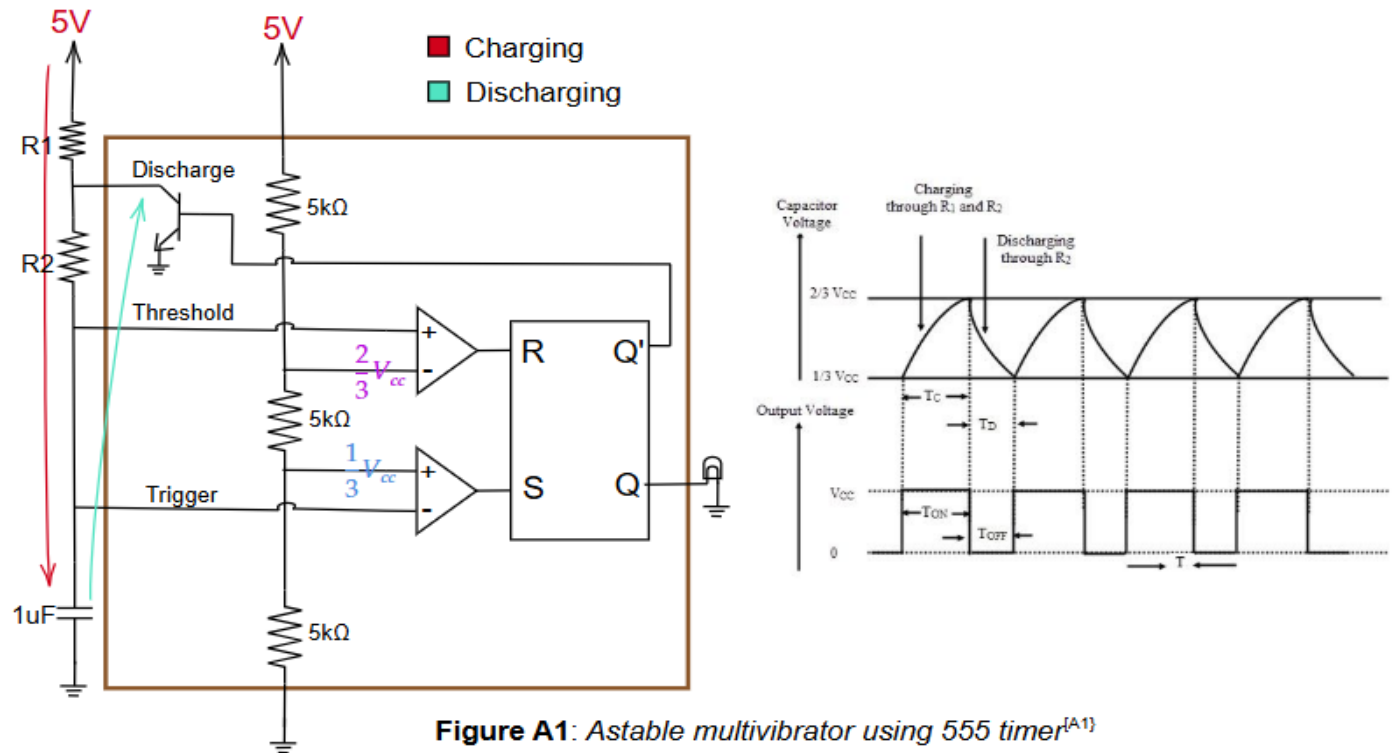


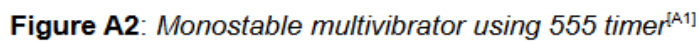
Figure A1: Astable multivibrator using 555 timer^[A1]

- Initially the SR latch is set and the capacitor charges from 0V to $\frac{1}{3}V_{cc}$. Both the magnitude comparators during this time are 0, maintaining SR latch in previous state, which is 1(LED is ON).
- When the capacitor reaches the threshold of $\frac{2}{3}V_{cc}$, the value of R and S becomes 1 and 0 respectively. It resets the SR latch which activates the transistor via base and the capacitor starts discharging through the transistor, now the LED is OFF.
- When discharging, both the magnitude comparators are outputting zero, latching the discharge state (LED is OFF).
- Once the capacitor reaches $\frac{1}{3}V_{cc}$ via discharging, it acts as a trigger by setting the SR latch, and deactivates the discharge transistor, hence once again charging the capacitor (LED is ON).
- Then this cycle is repeated to produce a astable pulse or a square wave.

Here the charging of the capacitor depends on both R1 and R2 whereas the discharging of the capacitor only depends on R2. To keep the charging time and discharging time fairly equal, the R1 is chosen in such a way that is smaller when compared to R2.

A.2 Monostable multivibrator working

In monostable multivibrator, one state is stable whereas the other state is unstable, i.e. it has only one stable state, hence the name monostable. In the astable multivibrator, the capacitor's voltage dropping below $\frac{2}{3}V_{cc}$ was the trigger for the timer, here the external

[illegible]

- $S = 0$
- When the capacitor is charging from 0V to $\frac{2}{3}V_{cc}$, $R=0$, maintaining the stable state 0, LED is OFF.
- Once it reaches $\frac{2}{3}V_{cc}$, $R=1$, activating the transistor thus discharging the capacitor. Note that there is no resistor between the capacitor and the transistor, which makes the discharging of the capacitor instantaneous. So, $R=1$ condition will not persist.

- $S = 1$
- When the capacitor is charging from 0V to $\frac{2}{3}V_{cc}$, $R = 0$, setting the SR latch, turning the LED ON. This is latched and other further trigger pulses are ignored for a time period that is dependent on the capacitance.

When the switch is released, S becomes 0(LED is OFF), the capacitor discharges immediately and this state is latched.

A.3 Bistable multivibrator working

A bistable multivibrator has two stable states, and switches between them based on an external trigger. Here the threshold pin is grounded, making $R = 0$ all the time. When the trigger pin is connected to ground, the falling edge of the trigger pulse sets the latch and LED is ON until the reset is grounded. When reset is grounded, we will get the output as 0 (LED is OFF). The reset pin of the 555 timer overrides the SR latch, until we switch the connection to ground from reset pin to trigger pin, it will keep on outputting 0.

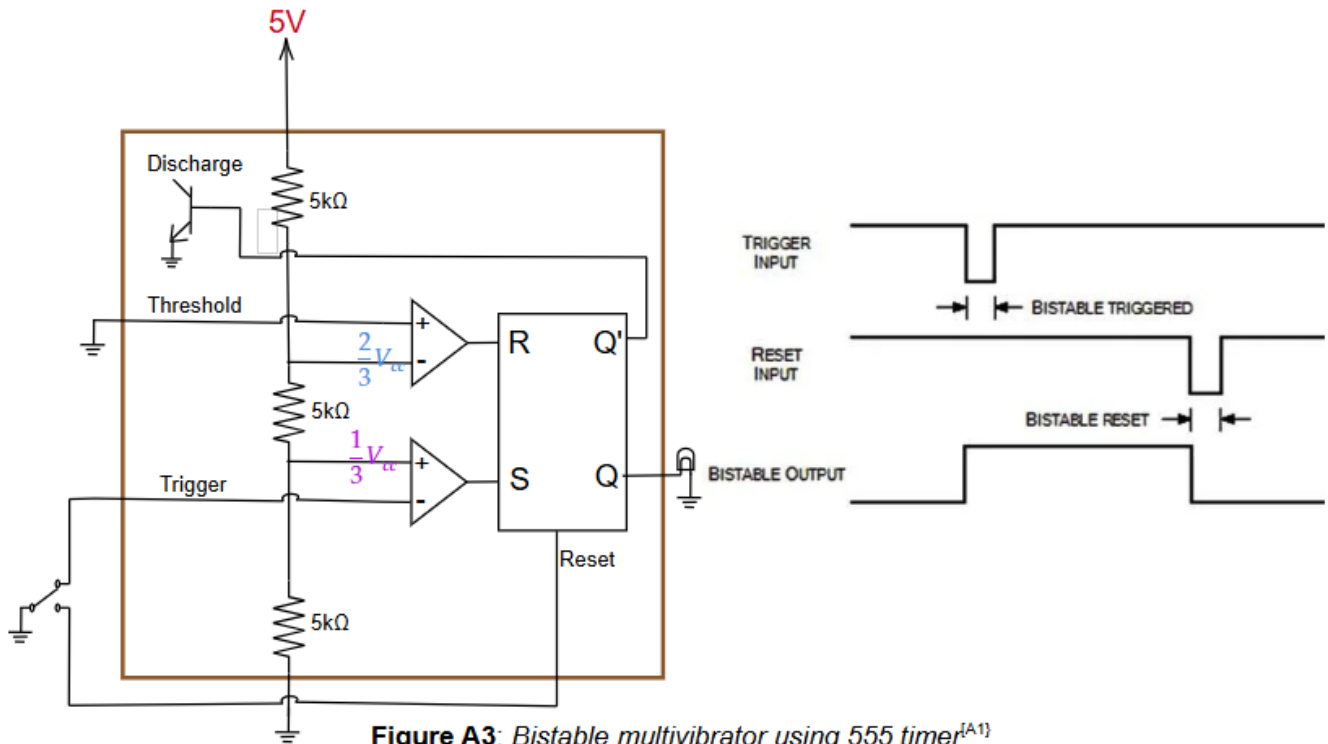


Figure A3: Bistable multivibrator using 555 timer^[A1]

B. D flip flop

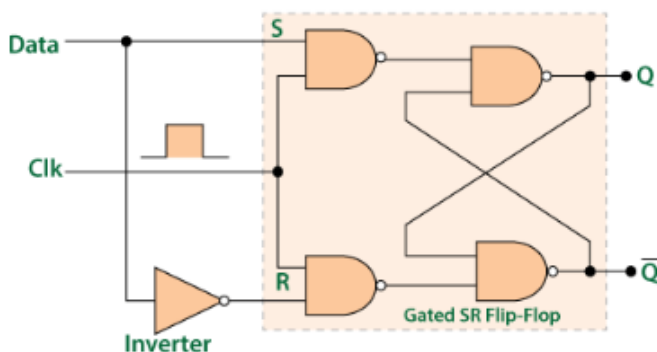


Figure B1: D flip flop^[B1]

Clock	D	Q	Q'	Description
↓ » 0	X	Q	Q'	Memory no change
↑ » 1	0	0	1	Reset Q » 0
↑ » 1	1	1	0	Set Q » 1

Figure B2: D flip flop truth table^[B2]

In D flip flop the data latches only during the rising edge of the clock. The clock acts as enable for the D flip flop, if I disable the clock the flip flop won't latch the data.

Along with this we require a functionality in which the data will only latch the data when there is a external trigger, let's call it Load. So even when the flip flop is enabled, only

when Load is HIGH, the data should be latched. It essentially works as an 1 bit register. The following figure is the circuit for the functionality.

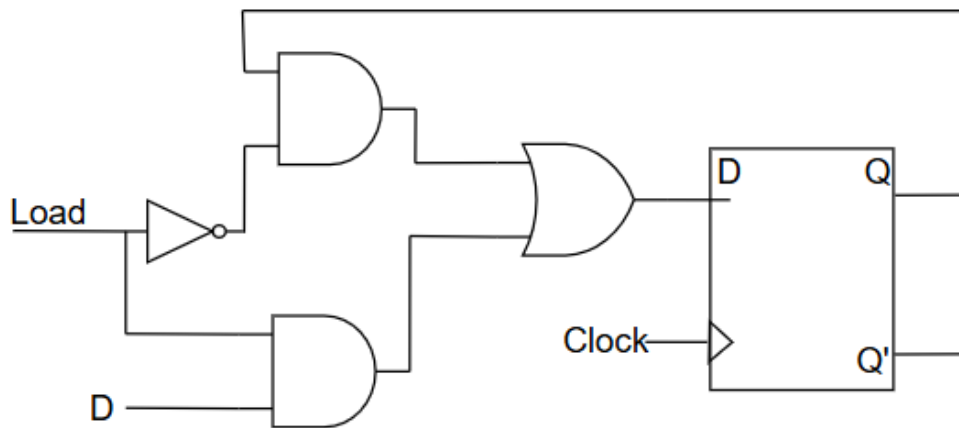


Figure B3: *D flip flop with Load functionality*

REFERENCES:

- [1] <https://eater.net/8bit>
- [2] <https://www.quisure.com/blog/faq/what-are-the-push-button-switches-commonly-used-by-electricians>
- [3] <https://electronicsbeliever.com/what-is-source-and-sink-current/>
- [4] 74LS173AN 4-bit D register datasheet
- [5] 74LS245 Octal bus transceiver datasheet
- [6] 74LS283 4-adder IC
- [7] <https://electronics.stackexchange.com/questions/43464/logic-gate-counter-on-7-segment-display>
- [8] AT28C16-15 16k parallel EEPROM datasheet
- [9] <https://electronics.stackexchange.com/questions/343734/improvements-to-7-segment-decoder>
- [10] 74HC595D 8 bit 3 state shift register
- [11] <https://github.com/beneater/eprom-programmer/blob/master/eprom-programmer/eprom-programmer.ino>
- [12] <https://github.com/beneater/eprom-programmer/blob/master/multiplexed-display/multiplexed-display.ino>

[13] <https://github.com/beneater/eprom-programmer/blob/master/microcode-eprom-programmer/microcode-eprom-programmer.ino>

[14] *Church-Turing thesis - Wikipedia*

[15] <https://github.com/beneater/eprom-programmer/blob/master/microcode-eprom-with-flags/microcode-eprom-with-flags.ino>

[A1] *xx555 Precision Timer datasheet - Texas Instruments*

[B1] <https://electronics.stackexchange.com/questions/684432/whats-the-difference-between-a-latch-and-a-flip-flop-i-found-multiple-answers>

[B2] <https://www.bartleby.com/questions-and-answers/t-which-moment-does-a-d-flip-flop-store-the-current-at-the-aforementioned-moment-value-of-d-during-/>